

Lecture No.23

Single right rotation

Is applied when a node is insert in right subtree.

ALGORITHM:

STEP 1: START.

STEP 2: INITIALIZE arr[] = {1, 2, 3, 4, 5 }.

STEP 3: length= sizeof(arr)/sizeof(arr[0])

STEP 4: SET n =3.

STEP 5: PRINT "Original Array"

STEP 6: SET i=0. REPEAT STEP 7 and STEP 8 UNTIL i<length.

STEP 7: PRINT arr[i]

STEP 8: i=i+1.

Single left rotation

Is applied when a node is inserted in a left subtree

ALGORITHM:

STEP 1: START.

STEP 2: INITIALIZE arr[] = {1, 2, 3, 4, 5 }.

STEP 3: length = sizeof(arr1)/sizeof(arr1[0])

STEP 4: length = sizeof(arr)/sizeof(arr[0])

STEP 5: SET n =3.

STEP 6: SET i=0. REPEAT STEP 7 to STEP 8 UNTIL i<length.

STEP 7: PRINT arr[i]

STEP 8: i = i+1.

A double right rotation, or right-left rotation, or simply RL, is a rotation that must be performed when attempting to balance a tree which has a left subtree, that is right heavy. This is a mirror operation of what was illustrated in the section on Left-Right Rotations, or double left rotations.

Double left right rotation Carries out single left rotation at first

Deleting a node from an AVL tree is similar to that in a binary search tree. Deletion may disturb the balance factor of an AVL tree and therefore the tree needs to be rebalanced in order to maintain the AVLness. For this purpose, we need to perform rotations

Lecture No.24

Other uses of binary trees

Binary Search Tree is a tree that allows fast search, insert, delete on a sorted data. It also allows finding closest item. Heap is a tree data structure which is implemented using arrays and used to implement priority queues. B-Tree and B+ Tree: They are used to implement indexing in databases.

An expression tree is a representation of expressions arranged in a tree-like data structure

Parse tree is the hierarchical representation of terminals or non-terminals. These symbols (terminals or non-terminals) represent the derivation of the grammar to yield input strings. ... The starting symbol of the grammar must be used as the root of the Parse Tree. Leaves of parse tree represent terminals

Parse tree for an SQL Query

A parse tree is a data structure for representing a parsed statement. Parsing a statement requires the grammar of the language (quest language, e.g., MySQL, MS-SQL, etc.) that the statement was written.

In computing , an optimizing compiler is a compiler that tries to minimize or maximize some attributes of an executable computer program. Common requirements are to minimize a program's execution time, memory footprint, storage size, and power consumption

Lecture No.25

An expression tree is a representation of expressions arranged in a tree-like data structure. In other words, it is a tree with leaves as operands of the expression and nodes contain the operators. Similar to other data structures, data interaction is also possible in an expression tree.

Huffman coding is a lossless data compression algorithm. The idea is to assign variable-length codes to input characters, lengths of the assigned codes are based on the frequencies of corresponding characters. The most frequent character gets the smallest code and the least frequent character gets the largest code.

Lecture No.26

*Huffman encoding*is used in data compression.

Mathematical properties of binary tree

The path length of a tree is the sum of the levels of all the tree's nodes. The internal path length of a binary tree is the sum of the levels of all the tree's internal nodes. The external path length of a binary tree is the sum of the levels of all the tree's external nodes.

Lecture No.27

Let's now focus on some basic properties of a binary tree:

A binary tree can have a maximum of nodes at level l if the level of the root is zero.

When each node of a binary tree has one or two children, the number of leaf nodes (nodes with no children) is one more than the number of nodes that have two children.

In computing, a threaded binary tree is a binary tree variant that facilitates traversal in a particular order. An entire binary search tree can be easily traversed in order of the main key, but given only a pointer to a node, finding the node which comes next may be slow or impossible.

A threaded insert, also known as a threaded bushing, is a fastener element that is inserted into an object to add a threaded hole.

In Binary Tree, Inorder successor of a node is the next node in Inorder traversal of the Binary Tree. ... In Binary Search Tree, Inorder Successor of an input node can also be defined as the node with the smallest key greater than the key of the input node

Inorder traversal

Definition: Process all nodes of a tree by recursively processing the left subtree, then processing the root, and finally the right subtree. Also known as symmetric traversal

Lecture No.28

Inorder traversal in threaded trees: inorder routine =left

/* The inorder routine for threaded binary tree */

```
TreeNode* nextInorder(TreeNode* p){
    if(p->RTH == thread) return(p->R);
    else {
        p = p->R;
        while(p->LTH == child)
            p = p->L;
        return p;
    }
}
```

}

}

Complete binary tree

A binary tree in which every level (depth), except possibly the deepest, is completely filled. At depth n , the height of the tree, all nodes must be as far left as possible.

Lecture No.29

The heap is an amorphous block of memory that your C++ program can access as necessary.

A max-heap is a complete binary tree in which the value in each internal node is greater than or equal to the values in the children of that node.

Insert -2 into a following heap:

Insert a new element to the end of the array:

In the general case, after insertion, heap property near the new node is broken:

To restore heap property, algorithm sifts up the new element, by swapping it with its parent:

Now heap property is broken at the root node:

Keep sifting:

Lecture No.30

Inserting into a min heap is always in the root of the tree.

Inserting into a max heap is Always lies in the root node

Deletion in Max (or Min) Heap always happens at the root to remove the Maximum (or minimum) value. Step 1 – Remove root node. Step 2 – Move the last element of last level to root. Step 3 – Compare the value of this child node with its parent. Step 4 – If value of parent is less than child, then swap them.

Building a heap

Convert an array into a heap by executing heapify progressively closer to the root. For an array of n nodes, this takes $O(n)$ time under the comparison model.

Lecture No.31

Other Heap Methods

`decreaseKey(p, delta)`

This method lowers the value of the key at position 'p' by the amount 'delta'. Since this might violate the heap order, so it (the heap) must be reorganized with percolate up (in min-heap) or down (in max-heap).

This method takes a pointer to the node that may be the array position as we are implementing it as an array internally. The user wants to decrease the value of this node by delt

`increaseKey(p, delta)`

This method is the opposite of decreaseKey. It will increase the value of the element by delta. These methods are useful while implementing the priority queues using heap.

`remove(p)`

This method removes the node at position p from the heap. This is done first by decreaseKey(p, $\ominus$) and then performing deleteMin(). First of all, we will decrease the value of the node by $\ominus$ and call the method deleteMin. The deleteMin method deletes the root. If we have a min-heap, the root node contains the smallest value of the tree. After deleting the node, we will use the percolateDown method to restore the order of the heap.

The user can delete any node from the tree.

Lecture No.32

`std::min` in C++

1: It compares the two numbers passed in its arguments and returns the smaller of the two, and if both are equal, then it returns the first one.

2: It can also compare the two numbers using a binary function, which is defined by the user, and then passed as argument in `std::min()`.

Build heap method takes an array along with its size

Following is the code of this method.

```
template <class eType>
```

```

void Heap<eType>::buildHeap(eType* anArray, int n )
{
    for(int i = 1; i <= n; i++)
        array[i] = anArray[i-1];
    currentSize = n;
    for( int i = currentSize / 2; i > 0; i-- )
        percolateDown( i );
}

```

build heap in linear time is better than "n"

Theorem

According to this theorem, “For a perfect binary tree of height h containing $2^{h+1} - 1$ nodes, the sum of the heights of nodes is $2^{h+1} - 1 - (h+1)$, or $N-h-1$ ”.

Lecture No.33

Priority queue is a type of queue in which every element has a key associated to it and the queue returns the element according to these keys, unlike the traditional queue which works on first come first serve basis.

Heap sort is a comparison-based sorting technique based on Binary Heap data structure.

♦ Disjoint set ADT: Used to represent a collection of sets. containing objects that are related to each other. ⇨ Relations defined through Union operation. ⇨ Union merges two sets – their objects become related.

An equivalence relation is a relationship on a set, generally denoted by “ $\sim$ ”, that is reflexive, symmetric, and transitive for everything in the set. ... Example: The relation “is equal to”, denoted “ $=$ ”, is an equivalence relation on the set of real numbers since for any $x, y, z \in \mathbb{R}$: 1.

Lecture No.34

Equivalence relation

‘A binary relation R over a set S is called an equivalence relation if it has following

properties’:

1. Reflexivity: for all element $x \in S$, $x R x$
2. Symmetry: for all elements x and y , $x R y$ if and only if $y R x$
3. Transitivity: for all elements x , y and z , if $x R y$ and $y R z$ then $x R z$

Disjoint set is basically as group of sets where no item can be in more than one set. It supports union and find operation on subsets. Find(): It is used to find in which subset a particular element is in and returns the representative of that particular set.

5:48

17.4 K/s 74

Dynamic Equivalence Problem

- The trees we will use are not necessarily binary.
- To perform union of two sets, we merge the two trees by making the root of one point to the root of the other.
- A *find*(x) on element x is performed by returning the root of the tree containing x .

Lecture No.35

Run-time analysis is a theoretical classification that estimates and anticipates the increase in running time (or run-time) of an algorithm as its input size (usually denoted as n) increases.

- union is clearly a constant time operation.
- Running time of *find*(i) is proportional to the height of the tree containing node i .

- This can be proportional to n in the worst case (but not always)
- Goal: Modify union to ensure that heights stay small

Imp point :not use pointer use array indices as pointer

Lecture No.36

Union by Size

Following are the salient characteristics of this method:

- Maintain sizes (number of nodes) of all trees, and during union.
- Make smaller tree, the subtree of the larger one.
- Implementation: for each root node i , instead of setting $\text{parent}[i]$ to -1 , set it to $-k$ if tree rooted at i has k nodes.
- This is also called union-by-weight.

Analysis of Union by Size

- If unions are done by weight (size), the depth of any element is never greater than $\log_2 n$.

Union by Height

- Alternative to union-by-size strategy: maintain heights,
- During union, make a tree with smaller height a subtree of the other.
- Details are left as an exercise.

Timing with Optimization

- Theorem: A sequence of m union and find operations, n of which are find operations, can be performed on a disjoint-set forest with union by rank (weight or height) and path compression in worst case time proportional to $(m\tilde{N}(n))$.
- $\tilde{N}(n)$ is the inverse Ackermann's function which grows extremely slowly.

For all practical purposes, $\tilde{N}(n) \approx 4$.

- Union-find is essentially proportional to m for a sequence of m operations, linear in m
-

Lecture No.37

In digital image processing and computer vision, image segmentation is the process of partitioning a digital image into multiple segments (sets of pixels, also known as image objects). ... Image segmentation is typically used to locate objects and boundaries (lines, curves, etc.) in images.

The definition of a maze is a system of paths that is difficult to navigate. An example of a maze is a labyrinth of tall corn stalks. ... Maze is defined as to daze or confuse.

Pseudo Code of the Maze Generation

We will use the entrance as 0 and exit as size-1

by default.

```
MakeMaze(int size) {  
    entrance = 0; exit = size-1;  
    while (find(entrance) != find(exit)) {  
        cell1 = randomly chosen cell  
        cell2 = randomly chosen adjacent cell  
        if (find(cell1) != find(cell2)) {  
            knock down wall between cells  
            union(cell1, cell2)  
        }  
    }  
}
```

After initializing the entrance and exit, we have a while loop. The loop will

Lecture No.38

Tables and Dictionaries

The table, an abstract data type, is a collection of rows and columns of information.

From rows and columns, we can understand that this is like a two dimensional array.

But it is not always a two dimensional array of same data type. Here in a table, the type of information in columns may be different. The data type of first column may be integer while the type of second column is likely to be string. Similarly there may be

other different data types in different columns. So the two-dimensional array used for table will have different data types.

A table consists of several columns, known as fields. These fields are some type of information. For example a telephone directory may have three fields i.e. name, address and phone number. On a computer system, the user account may have fields- user ID, password and home folder. Similarly a bank account may have fields like account number, account title, account type and balance of account.

Operations on Table ADT

insert

As the name shows this method is used to insert (add) a record in a table. For its execution, a field is designated as key. To insert a record (entry) in the table, there is need to know the key and the entry. The insert method puts the key and the other related fields in a table. Thus we add records in a table.

find

Suppose we have data in the table and want to find some particular information. The find method is given a key and it finds the entry associated with the key. In other words, it finds the whole record that has the same key value as provided to it. For example, in employees table if employee id is the key, we can find the record of an employee whose employee id is, say, 15466.

remove

Then there is the remove method that is given a value of the key to find and remove the entry associated with that key from the table.

Implementation of Table

implementation of the Table ADT depends on the answers to the following.

f How often entries are inserted, found and removed?

f How many of the possible key values are likely to be used?

f What is the likely pattern of searching for keys? Will most of the accesses be to just one or two key values?

f Is the table small enough to fit into the memory?

f How long will the table exist?

In a table for searching purposes, it is best to store the key and the entry separately (even though the key's value may be inside the entry)

Unsorted Sequential Array

In this implementation, we store the data of table in an array such that TableNodes are stored consecutively in any order. Each element of the row will have a key and entry of the record.

in unsorted sequential array, the insertion of data is fast but the find operation is slow and requires much time

A sorted array is an array data structure in which each element is sorted in numerical, alphabetical, or some other order, and placed at equally spaced addresses in computer memory. It is typically used in computer science to implement static lookup tables to hold multiple values which have the same data type.

insert

For the insertion of a new record in the array, we will have to insert it at a position in the array so that the array should be in sorted form after the insertion. We may need to shift the entries that already exist in the array to find the position of the new entry. For example, if a new entry needs to be inserted at the middle of the array, we will have to shift the entries after the middle position downward. Similarly if we have to add an entry at the start of the array, all the entries will be moved in the array to one position right (down). Thus we see that the insert operation is proportional to n (number of entries in the table). This means insert operation will take considerable time.

find

The find operation on a sorted array will search out a particular entry in $\log n$ time by the binary search. The binary search is a searching algorithm. Recall that in the tree, we also find an item in $\log n$ time. The same is in the case of sorted array.

remove

The remove operation is also proportional to n . The remove operation first finds the entry that takes $\log n$ time. While removing the data, it has to shuffle (move) the elements in the array to keep the sorted order. This shuffling is proportional to n . Suppose, we remove the first element from the array, then all the elements of the array have to be moved one position to left. Thus remove method is proportional to n .

Binary Search

The binary search is an algorithm of searching, used with the sorted data. As we have sorted elements in the array, binary search method can be employed to find data in the array. The binary search finds an element in the sorted array in $\log n$ time. If we have 100000 elements in the array, the $\log 100000$ will be 20 i.e. very small as compared to 100000. Thus binary search is very fast.

Lecture No.39

Binary search works on sorted arrays. Binary search begins by comparing an element in the middle of the array with the target value. If the target value matches the element, its position in the array is returned. If the target value is less than the element, the search continues in the lower half of the array.

Finding a specific member of an array means searching the array until the member is found. ... It compares each element with the value being searched for, and stops when either the value is found or the end of the array is encountered.

```
if ( value == middle element )
```

```
    value is found
```

```
else if ( value < middle element )
```

```
    search left half of list with the same method
```

```
else
```

```
    search right half of list with the same method
```

Binary Search – C++ Code

```
int isPresent(int *arr, int val, int N)
```

```
{
```

```

int low = 0;
int high = N - 1;
int mid;
while ( low <= high )
{
mid = ( low + high )/2;
if (arr[mid] == val)
return 1; // found!
else if (arr[mid] < val)
low = mid + 1;
else
high = mid - 1;
}
return 0; // not found

```

Binary Search - Efficiency

To see the efficiency of this binary search algorithm, consider when we divide the array of N items into two halves first time.

After 1 bisection $N/2$ items

After 2 bisections $N/4 = N/2^2$ items

...

After i bisections $N/2^i = 1$ item

$$i = \log_2 N$$

Implementation 3 (of Table ADT): Linked List

TableNodes are again stored consecutively (unsorted or sorted)

- insert: add to front; (1 or n for a sorted list)
- find: search through potentially all the keys, one at a time; (n for unsorted or for

a sorted list

- remove: find, remove using pointer alterations; (n)

Mcqs

when we used sorted array, the find operation was optimized

Implementation 4 (of Table ADT): Skip List

- Overcome basic limitations of previous lists
 - o Search and update require linear time
- Fast Searching of Sorted Chain
- Provide alternative to BST (binary search trees) and related tree structures.

Balancing can be expensive.

- Relatively recent data structure: Bill Pugh proposed it in 1990.
- Skip list contains a hierarchy of chains
- In general level i contains a subset of elements in level $i-1$ Skip list becomes a hierarchy of chains and every level contains a subset of element of previous level.

Using this kind of skip list data structure, we can find elements in $\log_2 n$ time. But the problem with this is that the frequency of pointers is so high as compared to the size of the data items that it becomes difficult to manage them. The insert and remove operations on this kind of skip list become very complex because single insertion or removal requires lot of pointers to readjust.

Professor Pugh suggested here that instead of doing leveling in powers of 2, it should be done randomly. Randomness in skip lists is a new topic for us. Let's see a formal definition of skip list

Skip List - Formally

- A skip list for a set S of distinct (key, element) items is a series of lists $S_0, S_1, \dots, S_h$ such that
 - o Each list S_i contains the special keys $+\infty$ and $-\infty$
 - o List S_0 contains the keys of S in non-decreasing order Each list is a subsequence of the previous one, i.e.,

$S_0 \supseteq S_1 \supseteq \dots \supseteq S_h$

o List S_h contains only the two special keys

Binary Search – Example 1

Case 1: $val == a[mid]$

$val = 10$

$low = 0, high = 8$

$mid = (0 + 8) / 2 = 4$

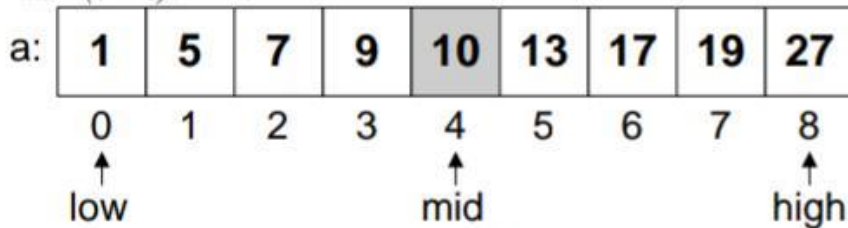


Fig 39.2

Binary Search – Example 2

Case 2: $val > a[mid]$

$val = 19$

$low = 0, high = 8$

$mid = (0 + 8) / 2 = 4$

$new\ low = mid + 1 = 5$

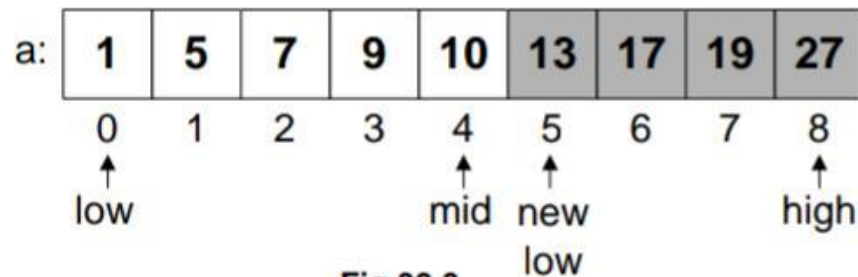


Fig 39.3

The second case is about the scenario when value (val) is greater than the middle value ($a[mid]$). The range of data items (low and $high$) is the same as that in case 1. Therefore, the middle position (mid) is also the same. But the value (val) 19 is greater than the value at the middle (mid) 10 of the array. As this array is sorted, therefore, the left half of the array must not contain value 19. At this point of time, our information about val 19 is that it is greater than the middle. So it might be present in the right half of the array. The right half part starts from position 5 to position 8. It is shown in Fig 39.3 that the *new low* is at position 5. With these new low and high positions, the algorithm is applied to this right half again.

Now, we are left with one more case.

Binary Search – Example 3

Case 3: $val < a[mid]$

$val = 7$

$low = 0, high = 8$

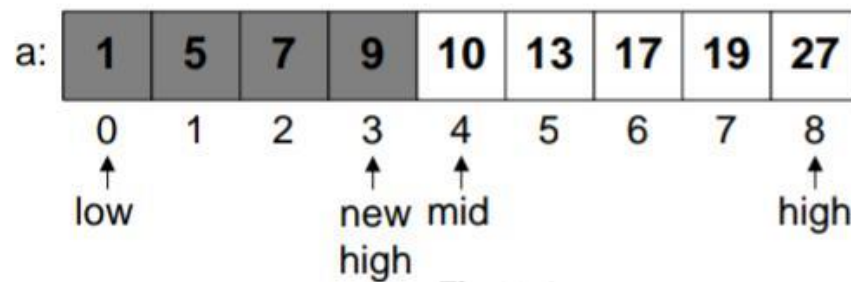
$mid = (0 + 8) / 2 = 4$

$new\ high = mid - 1 = 5$

Page 432 of 505

CS301 – Data Structures

Lecture No. 39



Lecture No.40

Skip List

A skip list for a set S of distinct (key, element) items is a series of lists $S_0, S_1, \dots, S_h$ such that

- Each list S_i contains the special keys $+\infty$ and $-\infty$
- List S_0 contains the keys of S in non-decreasing order
- Each list is a subsequence of the previous one, i.e.,

$S_0 \supseteq S_1 \supseteq \dots \supseteq S_h$

- List S_h contains only the two special keys

Skip list search

We search for a key x in the following fashion:

- We start at the first position of the top list
- At the current position p , we compare x with $y \leftarrow \text{key}(\text{after}(p))$
- $x = y$: we return $\text{element}(\text{after}(p))$
- $x > y$: we “scan forward”
- $x < y$: we “drop down”
- If we try to drop down past the bottom list, we return `NO_SUCH_KEY`

Insertion in Skip List

When we are going to insert (add) an item $(x,0)$ into a skip list, we use a randomized algorithm. Note that here we are sending the item in a pair. This is due to the fact that we keep the data and the key separately to apply the find and remove methods on table easily. In this pair, x is the key, also present in the record. The 0 denotes the data (i.e. whole record)

The first step of the algorithm is that

- We repeatedly toss a coin until we get tails, and we denote with i the number of times the coin came up heads.

second step of algorithm comes, stating that

- If $i > h$, we add to the skip list new lists $S_{h+1}, \dots, S_{i+1}$, each containing only the two special keys

The next steps are:

- We search for x in the skip list and find the positions $p_0, p_1, \dots, p_i$ of the items with largest key less than x in each list $S_0, S_1, \dots, S_i$
- For $j \leftarrow 0, \dots, i$, we insert item (x, o) into list S_j after position p_j

The

algorithms that use random numbers are generally known as randomized algorithms .

So

f A randomized algorithm performs coin tosses (i.e., uses random bits) to control its execution

f It contains statements of the type

$b \leftarrow \text{random}()$

if $b \leq 0.5$ // head

do A ...

else // tail

do B ...

f Its running time depends on the outcome of the coin tosses, i.e, head or tail

Deletion from a skip list is like deleting the same value from i different lists, where i is the number of levels the coin flip chose for this element.

Lecture No.41

Mcqs

three methods of skip list i.e. insert, find and remove

Quad node

A quadtree is a tree data structure in which each internal node has exactly four children. Quadtrees are the two-dimensional analog of octrees and are most often used to partition a two-dimensional space by recursively subdividing it into four quadrants or regions.

*In this method, we

do not have the array of pointers.* Rather, there are four next pointers. The following details can help us understand it properly.

A quad-node stores:

- item
- link to the node before

- link to the node after
- link to the node below
- link to the node above

Performance of skip list

In a skip list, with n items the expected space used is proportional to n . When we create a skip list, some memory is needed for its items. We also need memory to create a link list at lowest level as well as in other levels. This memory space is proportional to n

AVL tree

AVL tree is a self-balancing Binary Search Tree (BST) where the difference between heights of left and right subtrees cannot be more than one for all nodes. An Example Tree that is an AVL Tree. The above tree is AVL because differences between heights of left and right subtrees for every node is less than or equal to 1

Hashing

The hashing is an algorithmic procedure and a methodology. It is not a new data structure. It is a way to use the existing data structure.

In Hashing, we will internally use array. It may be static or dynamic. But we will not store data in consecutive locations. Their place of storage is calculated using the key and a hash function

find method . It will calculate the place of storage and retrieve the entry.

We will get the key and pass it to the hash function and obtain the array index. We get the data element from that array position. If data is not present at that array position, it means data is not found. We do not need to find the data at some other place. In case of binary search tree, we traverse the tree to find the element. Similarly in list structure we continue our search. Therefore find is also a constant time operation with Hashing.

Finally, we have remove method . It will calculate the place of storage and set it to null. That means it will pass the key to the hash function and get the array index.

Using this array index, it will remove the element.

Examples of hashing

HashCode ("apple") = 5

hashCode ("watermelon") = 3

hashCode ("grapes") = 8

hashCode ("cantaloupe") = 7

hashCode ("kiwi") = 0

hashCode ("strawberry") = 9

hashCode ("mango") = 6

hashCode ("banana") = 2

table[5] = "apple"

table[3] = "watermelon"

table[8] = "grapes"

table[7] = "cantaloupe"

table[0] = "kiwi"

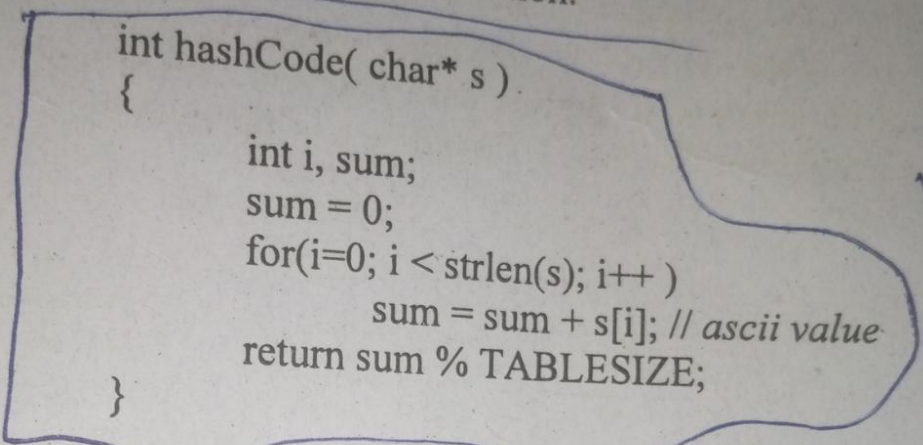
table[9] = "strawberry"

table[6] = "mango"

table[2] = "banana"

Let's see the C++ code of *hashCode* function.

```
int hashCode( char* s )
{
    int i, sum;
    sum = 0;
    for(i=0; i < strlen(s); i++ )
        sum = sum + s[i]; // ascii value
    return sum % TABLESIZE;
}
```

A photograph of a piece of paper with handwritten C++ code. The code is enclosed in a hand-drawn blue rectangular box. Above the box, the text "Let's see the C++ code of hashCode function." is written. To the right of the box, there is a small blue checkmark. The code itself is: int hashCode(char* s) { int i, sum; sum = 0; for(i=0; i < strlen(s); i++) sum = sum + s[i]; // ascii value return sum % TABLESIZE; }

$$h(str) = \left(\sum_{i=0}^{length-1} str[i] \times b^i \right) \% T$$

$$Example: h(ABC) = (65b^0 + 66b^1 + 67b^2) \% T$$

$$h(str) = \left(\sum_{i=0}^{length-1} str[i] \right) \% TableSize$$

$$Example: h(ABC) = (65 + 66 + 67) \% TableSize$$

Lecture No.42

The definition of collision is:

“When two values hash to the same array location, this is called a collision”

We cannot say that the usage of this hash function will not result in collision especially when the data is changing. Collisions are normally treated as a phenomenon of “first come, first served”, the first value that hashes to the location gets i

Linear Probing

The first solution is known as open addressing. When there is a collision, we try to find some other place in our array. This approach of handling collisions is called open addressing; it is also known as closed hashing. The word open is used with addressing and the word closed is used with hashing. Be careful when naming these.

More formally, cells at $h_0(x)$, $h_1(x)$, $h_2(x)$, ... are tried in succession where

$h_i(x) = (\text{hash}(x) + f(i)) \bmod \text{TableSize}$, with $f(0) = 0$.

Here hash is our hash function. If there is some collision, we add $f(i)$ value to it before taking its mod with TableSize. The function, f , is the collision resolution strategy.

We use $f(i) = i$, i.e., f is a linear function of i . Thus

$\text{location}(x) = (\text{hash}(x) + i) \bmod \text{TableSize}$

The collision resolution strategy is called linear probing as it scans the array sequentially (with wrap around) in search of an empty cell

*In

linear probing, at the time of collisions, we add one to the index and check that location. If it is also not empty, we add 2 and check that position*

Linear probing and Deletion

delete procedure. If an item is placed in $\text{array}[\text{hash}(\text{key})+4]$, the item just before it is deleted. How will probe determine that the “hole” does not indicate the item is not in the array? We may have three states for each location as:

- Occupied
- Empty (never used)
- Deleted (previously used)

Using the linear probe, we insert data where we get some empty space in the array. Firstly, we try to insert it at the index position given by the hash function. If it is occupied, we move to the next position and so on. In this way, we have a chain to follow. Now if an element of the chain is deleted, how can we know that it was filled previously

Clustering

One problem with linear probing technique is the tendency to form “clusters”. A cluster is a group of items not containing any open slots. The bigger a cluster gets, the more likely the new values will hash into the cluster, and make it even bigger.

Clusters cause efficiency to degrade.

The data is getting gathered

instead of scattering because linear probing inserts the data in the next position. It seems as the normal use of the array in which we insert data in the array from first position then next position and so on. It may depend on our data or our hash function. This gathering of data is called clustering.

We were trying to store the data in the array in a constant time or in a single step.

Similarly the find and remove methods should be of constant time. That attribute has now been lost. What should we do now? One of the solutions is quadratic probing.

Quadratic probing uses different formula:

f Use $F(i) = i^2$

(square of i) to resolve collisions

f If hash function resolves to H and a search in cell H is inconclusive, try $H +$

1^2

, $H + 2^2$

, $H + 3^2$

, ...

f Advantages over open addressing:

- Simpler insertion and removal
- Array size is not a limitation

f Disadvantage

- Memory overhead is large if entries are small.

The problem in linear probing is that when our array is full what we should do. This problem can be solved using the link list.

Collision

Collision takes place when two or more keys (data items) produce the same index.

Page 459 of 505

CS301 – Data Structures

Lecture No. 42

Let's see the previous example of storing the names of fruits. Suppose our hash function gives us the following values:

hash("apple")	=	5	0	kiwi
hash("watermelon")	=	3		
hash("grapes")	=	8	1	
hash("cantaloupe")	=	7		
hash("kiwi")	=	0	2	banana
hash("strawberry")	=	9		
hash("mango")	=	6	3	watermelon
hash("banana")	=	2	4	
			5	apple
			6	mango
			7	cantaloupe
			8	grapes
			9	strawberry

Lecture No.43

Hashing animation

hashing animation . This animation will be shown in the browser. This is an applet written in java language. We will see linear probing, quadratic probing and link list chaining in it. This is an example of how do we solve collision. collision strategies in hashing. We studied three solutions, Linear Probing, Quadratic Probing and Linked List chaining. Hashing is vast research field, which covers hash functions, storage and collision issues etc. At

the moment, we will see hashing in implementation of table ADT. Operations of insert, delete and find are performed in constant time using this hashing strategy. Constant time means the time does not increase with the increase in data volume. However, if collisions start happening then the time does not remain constant. Especially if we see linear probing, we had to insert the data by sorting the array sequentially. Similar was the case with quadratic. In case of linked list, we start constructing linked list that takes time and memory. But later we will see some situations where hashing is very useful.

Today, we will study these three strategies of hash implementation using animations. These animations will be provided to you in a Java program. It is important to mention here that all the data structures and algorithms we have studied already can be implemented using any of the languages of C/C++ or Java. However, it is an important decision to choose the programming language because every language has its strong area, where it has better application. Java has become very popular because of its facilities for Internet. As you already know C++, therefore, Java is easy to learn for you. The syntax is quite similar. If we show you the java code you will say it is C++.

Applications of Hashing

Let's see few examples of those applications where hashing is highly useful. The hashing can be applied in table ADT or you can apply hashing using your array to store and retrieve data.

- Compilers use hash tables to keep track of declared variables (symbol table).

Another usage of hashing is given below:

- A hash table can be used for on-line spelling checkers — if misspelling detection (rather than correction) is important, an entire dictionary can be hashed and words checked in constant time.

*– *Game playing programs use hash tables to store seen positions, thereby saving computation time if the position is encountered again**

When Hashing is Suitable?

- Hash tables are very good if there is a need for many searches in a reasonably stable table.
- Hash tables are not so good if there are many insertions and deletions, or if table traversals are needed — in this case, AVL trees are better.

In some applications, it is required to frequently read and write data. In these kinds of applications hash table might not be a good solution, AVL tree might be a good option. But bear in mind that there are no hard and fast statistics to go for hash table and then to AVL tree. You have to be a good software engineer to choose relevant data structure.

- Also, hashing is very slow for any operations which require the entries to be sorted

- o e.g. Find the minimum key

Sorting

Sorting means to put the data in a certain order or sequence.

when we traverse

the binary search tree in in-order way, the obtained data happens to be sorted.

Similarly, we saw other data structures, where we used to keep data in sorted order. In case of min-heap if we keep on removing elements one by one, we get data in sorted order.

Sorting is so useful that in 80-90% of computer applications, sorting is there in one form or the other. Normally, sorting and searching go together. Lot of research has been done on sorting; you can get lot of stuff on it from different sources. Very efficient algorithms have already been developed for it. Moreover, a vast Mathematical analysis has been performed of these algorithms. If you want to expose yourself, how these analyses are performed and what Mathematical tools and procedures are employed for performing analysis then sorting is very useful topic for you.

Elementary Sorting Algorithms

- Selection Sort
- Insertion Sort
- Bubble Sort

These algorithms have been put as elementary because these are very simple. They will act as our baseline and we will compare them with other algorithms in order to find a better algorithm.

Selection Sort

- Main idea:
 - o find the smallest element
 - o put it in the first position
 - o find the next smallest element
 - o put it in the second position
 - ...
- And so on, until you get to the end of the list

This technique is so simple that you might have found it yourself already. You search the whole array and find the smallest number. The smallest number is put on the first position of the array while the previous element in this position is moved somewhere else. Find the second smallest number and then put that number in the second position in the array, again the previous number in that position is shifted somewhere else. We keep on performing this activity again and again and eventually we get the array sorted. This technique is called selection sort because we select elements for their sorted positions.

Lecture No.44

There are three elementary sorting methods being discussed in this hand out. These are- selection sort, insertion sort and bubble sort
known as the in- place sorting algorithm as there is no

need of additional storage to carry out this sorting.

Insertion Sort

The main idea of insertion sort is

- Start by considering the first two elements of the array data. If found out of order, swap them
- Consider the third element; insert it into the proper position among the first three elements.
- Consider the fourth element; insert it into the proper position among the first four elements.
-

Thus in this algorithm, we keep the left part of the array sorted and take element from the right and insert it in the left part at its proper place. Due to this process of insertion, it is called insertion sorting.

Following is the code of the insertion sort in C++.

```
void insertionSort(int *arr, int N)
{
    int pos, count, val;
    for(count=1; count < N; count++)
    {
        val = arr[count];
        for(pos=count-1; pos >= 0;
pos--)
            if (arr[pos] > val)
                arr[pos+1]=arr[pos];
        else break;

        arr[pos+1] = val;
    }
}
```

```
}
```

In bubble sort algorithm, we do not search the array for the smallest number like in the other two algorithms. Also we do not insert the element by shifting the other elements. In this algorithm, we do pair-wise swapping. We will take first the elements and swap the smaller with the larger number. Then we do the swap between the next pair. By repeating this process, the larger number will be going to the end of the array and smaller elements come to the start of the array.

Following is the code of bubble sort algorithm in C++.

```
void bubbleSort(int *arr, int N)
```

```
{
```

```
    int i, temp, bound = N-1;
```

```
    int swapped = 1;
```

```
    while (swapped > 0 )
```

```
    {
```

```
        swapped = 0;
```

```
        for(i=0; i < bound; i++)
```

```
            if ( arr[i] > arr[i+1] )
```

```
            {
```

```
                temp = arr[i];
```

```
                arr[i] = arr[i+1];
```

```
                arr[i+1] = temp;
```

```
                swapped = i;
```

```
            }
```

```
            bound = swapped;
```

```
        }
```

```
    }
```

Divide and Conquer

What if we split the list into two parts?

10	12	8	4	2	11	7	5
----	----	---	---	---	----	---	---

Divide and Conquer

Sort the two parts:

4	8	10	12	2	5	7	11
---	---	----	----	---	---	---	----

N	N²	N Log₂ (N)
10	100	33.21
100	10000	664.38
1000	1000000	9965.78
10000	100000000	132877.12
100000	10000000000	1660964.04
1000000	1E+12	19931568.57

Summary

❑ Insertion, Selection and Bubble sort:

- Worst case time complexity is proportional to N^2 .

Best sorting routines are $N \log(N)$

NLogN Algorithms

❑ Divide and Conquer

❑ Merge Sort

❑ Quick Sort

❑ Heap Sort

Bubble Sort Analysis

- ❑ What is the time complexity of this algorithm?
- ❑ Worst case > Average case > Best case
- ❑ Each iteration compares all the adjacent elements, swapping them if necessary

- first iteration $N - 1$ +
- second iteration $N - 2$ +
- ...
- last iteration 1

- Total $\frac{N(N-1)}{2} = O(N^2)$

Bubble Sort

- ❑ Basic idea (*lighter bubbles rise to the top*):
 - Exchange neighbouring items until the largest item reaches the end of the array
 - Repeat for the rest of the array

Insertion Sort Analysis

- ❑ What is the time complexity of this algorithm?
- ❑ Worst case > Average case > Best case
- ❑ Each iteration inserts an element at the start of the array, shifting all sorted elements along

- second element 2 +
- ...
- penultimate element N-1 +
- last element N

- Total $(2+N)(N-1)/2 = O(N^2)$

Selection Sort: Code

```
void selectionSort(int *arr, int N)
{
    int posmin, count, tmp;
    for(count=0;count<N;count++)
    {
        posmin = findIndexMin(arr,count,N);

        tmp=arr[posmin];
        arr[posmin]=arr[count];
        arr[count]=tmp;
    }
}
```

Selection Sort: Code

```
int findIndexMin(int *arr, int start,
                int N)
{
    int posmin=start;
    int index;
    for(index=start; index < N; index++)
        if (arr[index]<arr[posmin])
            posmin=index;

    return posmin;
}
```

Selection Sort Analysis

□ What is the time complexity of this algorithm?

Worst case == Best case == Average case

Each iteration performs a linear search on the rest of the array

first element N +

second element $N-1$ +

...

penultimate element 2 +

last element 1

Total $N(N+1)/2$

$$\frac{N(N+1)}{2} = \frac{(N^2+N)}{2}$$

Lecture No.45

Divide and conquer

three new sorting algorithms; merge

sort, quick sort and heap sort. All of these three algorithms take time proportional to $n \log_2 n$. Our elementary three sorting algorithms were taking n^2

n^2

time; therefore, these

new algorithms with $n \log_2 n$ time are faster. In search operation, we were trying to reduce the time from n to $\log_2 n$.

Let's see few analysis to confirm the usefulness of the divide and conquer technique.

- To sort the halves approximate time is $(n/2)^2 + (n/2)^2$
- To merge the two halves approximate time is n
- So, for $n=100$, divide and conquer takes approximately:

$$= (100/2)^2 + (100/2)^2 + 100$$

$$= 2500 + 2500 + 100$$

$$= 5100 \quad (n^2 = 10,000)$$

We know that elementary three sorting algorithms were taking approximately n^2

time.

time.

Suppose we are using insertion sort of those elementary algorithms. We divide the list into two halves then the time will be approximately $(n/2)^2 + (n/2)^2$

Mergesort

- Mergesort is a divide and conquer algorithm that does exactly that.
- It splits the list in half
- Mergesorts the two halves
- Then merges the two sorted halves together
- Mergesort can be implemented recursively

Let's see the mergesort algorithm, how does that work.

- The mergesort algorithm involves three steps:
 - o If the number of items to sort is 0 or 1, return
 - o Recursively sort the first and second halves separately
 - o Merge the two sorted halves into a sorted group
- If the data is consisting of 0 or 1 element then there is nothing required to be done further. If the number of elements is greater than 1 then apply divide and conquer strategy in order to sort them. Divide the list into two halves and sort them separately ...

Mergesort and Linked Lists

Merge sort works with arrays as well as linked lists. Now we see, how a linked list is sorted. Suppose we have a singly linked list as shown in figure . We can

divide the list into two halves as we are aware of the size of the linked list. Each half is processed recursively for sorting. Both of the sorted resultant halves are merged together.

Sort

- Mergesort is $O(n \log^2 n)$
- Space?
- The other sorts we have looked at (insertion, selection) are in-place (only require a constant amount of extra space)
- Mergesort requires $O(n)$ extra space for merging

Quicksort

- Quicksort is another divide and conquer algorithm.
- Quicksort is based on the idea of partitioning (splitting) the list around a pivot or split value.

Quicksort is also a divide and conquer algorithm.

Quicksort

```
void quickSort(int array[], int size)
{
    int index;

    if (size > 1)
    {
        index = partition(array, size);
        quickSort(array, index);
        quickSort(array+index+1, size - index-1);
    }
}
```

Quicksort

```
int partition(int array[], int size)
{
    int k;
    int mid = size/2;
    int index = 0;

    swap(array, array+mid);
    for (k = 1; k < size; k++){
        if (array[k] < array[0]){
            index++;
            swap(array+k, array+index);
        }
    }
    swap(array, array+index);
    return index;
}
```

Mergesort

Split the list in half. Mergesort the left half.

10	4	8	12	11	2	7	5
----	---	---	----	----	---	---	---

Split the list in half. Mergesort the left half.

10	4	8	12
----	---	---	----

Split the list in half. Mergesort the left half.

10	4
----	---

Mergesort the right.

10	4
----	---

Mergesort Analysis

Merging the two lists of size $n/2$:

$O(n)$



Merging the four lists of size $n/4$:

$O(n)$



Merging the n lists of size 1:

$O(n)$



$O(\lg n)$
times

- ☐ Mergesort is $O(n \lg n)$
- ☐ Space?
- ☐ The other sorts we have looked at (insertion, selection) are **in-place** (only require a constant amount of extra space)
- ☐ Mergesort requires $O(n)$ extra space for merging

Divide and Conquer

- ❑ Why not divide the halves in half?
- ❑ The quarters in half?
- ❑ And so on . . .
- ❑ When should we stop?

At $n = 1$

Divide and Conquer

Recall: Binary Search

